

INTRODUCTION

Access to quality education in Sierra Leone continues to be affected by overcrowded classrooms, a shortage of learning materials, poor tracking of student attendance and performance, and limited access to educational resources in rural and underserved communities. These challenges contribute directly to high dropout rates, widening learning gaps, and unequal educational outcomes between urban and rural schools. As a group, we set out to address this problem through the design of the Salone Learn Hub (SLH), a digital school management and e-learning platform intended to streamline how students are enrolled, taught, assessed, and supported throughout their academic journey.

This report presents the complete software design for SLH as a Digital Public Good, developed in line with the Software Development Life Cycle (SDLC) principles of the Agile model. It covers our choice of development methodology, the system's architecture and how its components interact, a fully designed Figma prototype of the system's key screens, our project plan (Gantt chart and network diagram), and our licensing, design quality, and Sustainable Development Goal (SDG) considerations. Our system is designed in direct alignment with SDG 4: Quality Education, with the goal of making learning more accessible, organized, and effective for both students and teachers across Sierra Leone.

SOFTWARE DEVELOPMENT LIFE CYCLE

For the development of SLH, we selected the Agile model as our Software Development Life Cycle methodology. We chose Agile for the following reasons:

We worked with a limited budget as a student group, and Agile is well suited for projects where resources are constrained. Although our project involved a significant amount of work covering system architecture, UI/UX prototyping in Figma, UML diagrams, project planning, licensing, and full documentation we were required to deliver all of this within a fixed academic timeframe ending at Week 11. Agile's sprint-based structure allowed us to manage this workload systematically, tackling one area at a time without compromising the quality of any deliverable. The Salone Learn Hub also had multiple features that were best built and tested one at a time rather than all at once, and as a group we needed a model that encouraged collaboration, shared responsibility, and continuous feedback all of which are core to how Agile works.

HOW WE DEVELOPED SLH USING AGILE

We developed the Salone Learn Hub in short, focused sprints. Rather than designing and building everything at once, we completed one set of features per sprint, reviewed and tested it as a group, fixed any issues, and then moved on to the next. This meant that at the end of each sprint we always had a working version of part of the system. We used GitHub throughout to track changes, collaborate, and store every version of our work. Figma handled our UI/UX design, and all documentation was updated continuously rather than left to the end.

AGILE STAGES AND WHAT HAPPENED IN EACH

Planning: As a group we defined the problem SLH was built to solve overcrowded classrooms, poor tracking of student performance, limited access to learning materials, and unequal educational opportunities in Sierra Leone's schools.

We identified all the features the system needed, assigned roles among ourselves, chose our tools, and mapped out a sprint schedule running from Week 4 to Week 11.

Design: We designed the overall structure of the Salone Learn Hub using a four-layer architecture covering the presentation layer, communication layer, application logic, and data layer. We also produced our Figma prototype at this stage, creating over ten screens representing how SLH looks and feels for real users across the student, teacher, parent, and administrator roles.

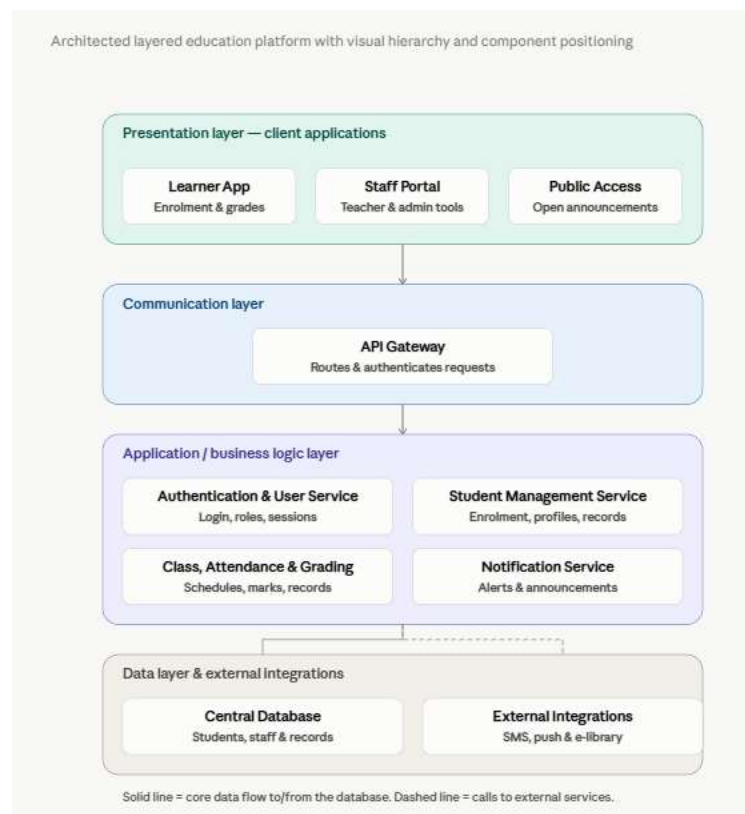
Development: We built SLH sprint by sprint. Sprint 1 covered student registration and enrolment. Sprint 2 handled class scheduling and learning material uploads. Sprint 3 brought the teacher and admin dashboards together, covering attendance and grading. Sprint 4 integrated everything through the parent portal and finalized the notification flow. After each sprint we reviewed what had been built as a group before moving forward.

Testing: Testing was carried out continuously after every sprint rather than saved for the end. Each feature of SLH was checked to make sure it worked as expected, bugs were identified and fixed immediately, and any improvements were carried into the next sprint.

Deployment: All project files including our report, Figma prototype, UML diagrams, and project plan were uploaded to our shared GitHub repository. This served as both our collaboration platform throughout the project and our final delivery point for SLH.

Maintenance: While the Salone Learn Hub is currently a prototype, we have already identified future improvements such as offline access to learning materials, multi-school support, and a live database backend all of which would be released as future sprints if SLH were deployed in a real school setting.

SYSTEM ARCHITECTURE DESIGN



SYSTEM ARCHITECTURE DESIGN — How the Components Interact

Our system, the Salone Learning Hub (SLH), is built around four layers, and we designed it this way so each part has a clear job and the whole system stays organized and easy to scale.

1. Presentation Layer — Learner App, Staff Portal, and Public Access

These are the three entry points designed into our Figma prototype. Our Learner App handles sign-up/login, the student dashboard, class schedule, grades, assignments, and notifications.

Our Staff Portal lets Teachers, Parents, and Administrators log in by role and use their own dashboards — marking attendance, entering grades, registering students, and viewing reports. We also built a Public Access path covering everything reachable without logging in: the splash screen, the Announcements button, and the About page. This is why on the home screen the Announcements option is always available even before a user signs up or logs in — we wanted anyone, including prospective parents or guardians, to reach essential school information immediately.

2. Communication Layer — API Gateway

Every request from either the Learner App or the Staff Portal passes through a single API Gateway before reaching our backend. The Gateway checks the user's session/login token, ensures they only access what their role allows (for example, a Parent cannot open the Teacher's grading screen), and forwards the request to the correct service. We chose this approach because it makes our architecture scalable — if we wanted to add another client later, such as a USSD/SMS version for guardians without smartphones, it could connect through the same Gateway without changing anything in the services behind it.

3. Application/Business Logic Layer — Our Four Core Services

This is where the actual logic of our system lives, split into four services:

Authentication & User Service – validates sign-up and login, manages 2FA and password resets, and enforces role-based permissions across Student, Teacher, Parent, and Admin accounts.

Student Management Service – handles student enrolment, search, profile/history, and the guardian and student-detail forms designed in our prototype (Registration Form 1 and Form 2).

Class, Attendance & Grading Service – manages class schedules, powers the "Mark Attendance" action on the Teacher dashboard, and handles grade entry and report-card generation.

Notification Service – sends alerts such as assignment due-date reminders, grade postings, attendance alerts, and school announcements to the Notifications screen.

These services do not talk to each other directly — they interact through the Gateway and the shared database. For example, when a Teacher marks attendance, the Class, Attendance & Grading Service saves the record, and the Notification Service immediately alerts any parent linked to a student marked absent.

4. Data Layer & External Integrations

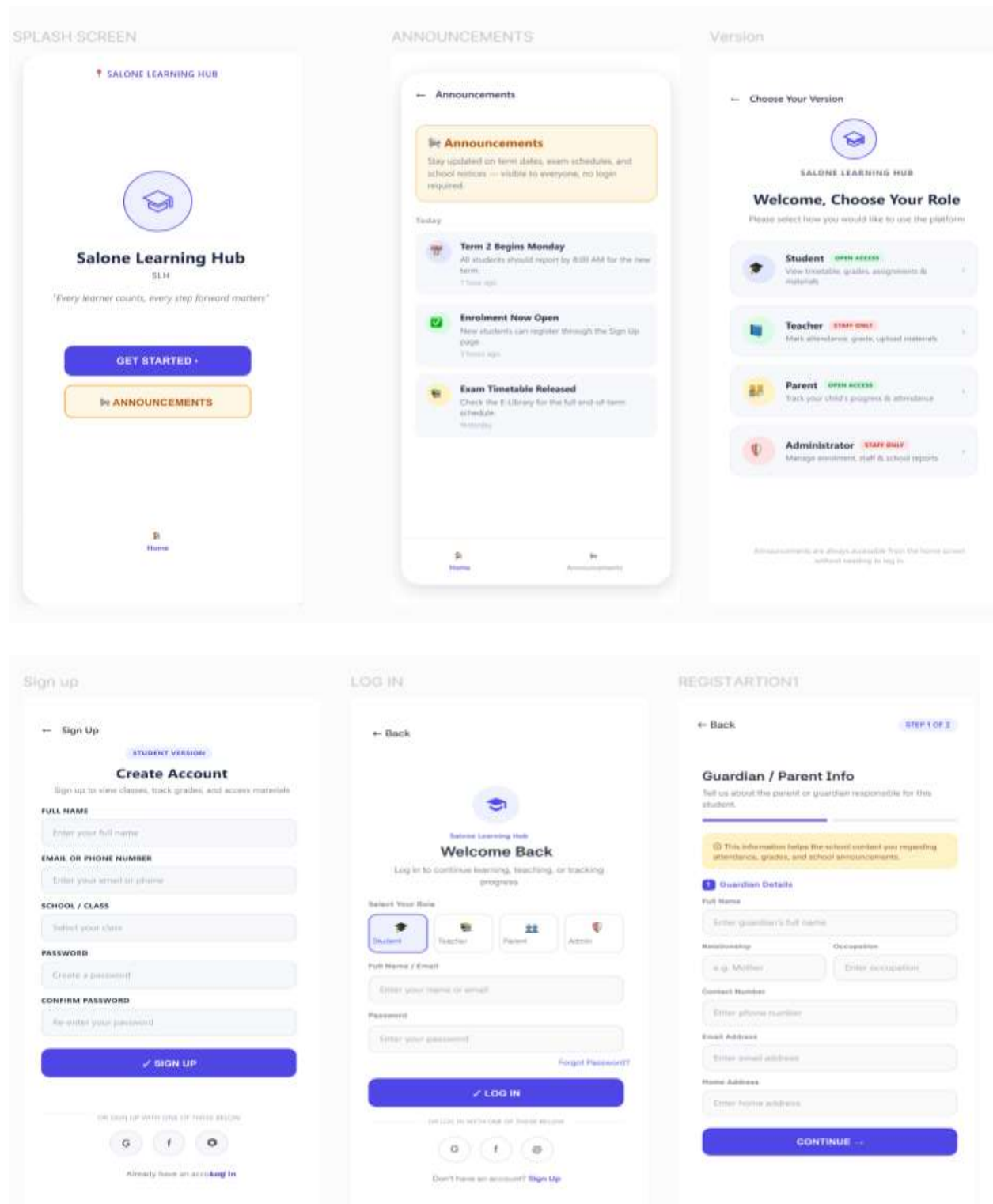
All four services read from and write to one Central Database, storing students, staff/roles, class records, grades, and the notification log. This centralization means that when a Teacher submits a grade, the Student's "My Grades" screen updates from the same source of truth the Parent Dashboard is reading from. Our system also connects to External Integrations: an SMS/push provider to deliver notifications, and an e-library service to host and serve learning materials.

A worked example demonstrating the flow end-to-end: a student opens the app → the Authentication Service verifies their login → the Student Dashboard requests grade data from the Class, Attendance & Grading Service through the Gateway → the service reads the record from the database → the Notification Service pushes a "Grade Posted" alert straight back to the screen the student is already looking at.

This is how our four layers and four services work together as one connected system rather than separate disconnected parts.

PROTOTYPE DEVELOPMENT

Figma Screenshots below:



REGISTRATION 2

[← Back](#)
STEP 2 OF 3

Student Details

Now let's fill in the student's personal and school information.

Student Details

Student's Full Name

Date of Birth

Gender

Class / Grade

Previous School

Home Address

Medical Notes (Optional)

Upload Student's Photo

[✓ SUBMIT FORM](#)

Get back and edit question info

Registration success

[← Registration](#)

Registration Submitted

The student has been successfully registered. An account confirmation has been sent to the guardian's contact.

REGISTRATION SUMMARY

Student Name	Aminata Kamara
Class	JSS 2B
Student ID	SFU-2025-0412
Guardian	Sallieu Kamara

[GO TO DASHBOARD](#)

[REGISTER ANOTHER STUDENT](#)

Forgot password

[← Forgot Password](#)

Reset Your Password

Enter the email or ID linked to your account and we'll send you a link to reset your password.

EMAIL / STUDENT ID

[SEND RESET LINK](#)

If you don't receive an email within a few minutes, check your spam folder or contact your school administrator.

Remembered your password? [Log In](#)

Student Dashboard

WELCOME LEARNING HUB

Hello, Aminata 🌟

JSS 2B • Next Class Mathematics, 10:30 AM

Quick Actions

Timetable

View class schedule

Grades

Check report card

E-library

Access digital resources

Assignments

Managing tasks

94% Attendance

B+ Avg Score

3 Due Dates

Upcoming

English Assignment

Due tomorrow, 5:00 PM

[Due Date](#)

Integrated Science Test

Today, 8:00 AM

[Reschedule](#)

[Dashboard](#)

[Classes](#)

[Library](#)

[Profile](#)

Parent Registration 1

[← Register Student](#)
STEP 1 OF 2

Guardian / Parent Info

Tell us about the guardian responsible for this student.

GUARDIAN DETAILS

FULL NAME

RELATIONSHIP

CONTACT NUMBER

EMAIL ADDRESS

HOME ADDRESS

[CONTINUE →](#)

Parent Registration 2

[← Register Student](#)
STEP 2 OF 2

Student Details

Now let's fill in the student's own school information.

STUDENT DETAILS

STUDENT'S FULL NAME

DATE OF BIRTH

GENDER

CLASS / GRADE

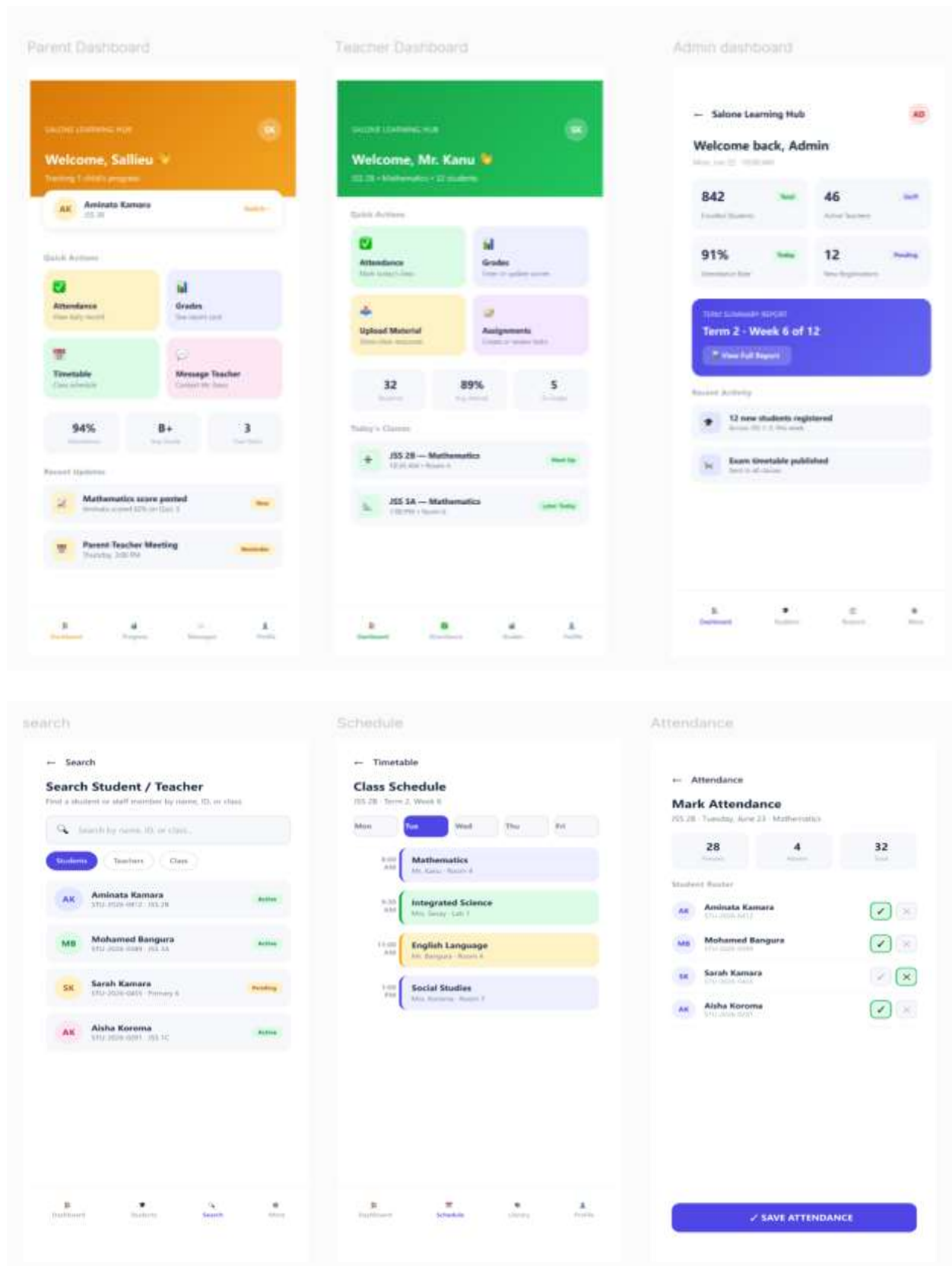
PREVIOUS SCHOOL

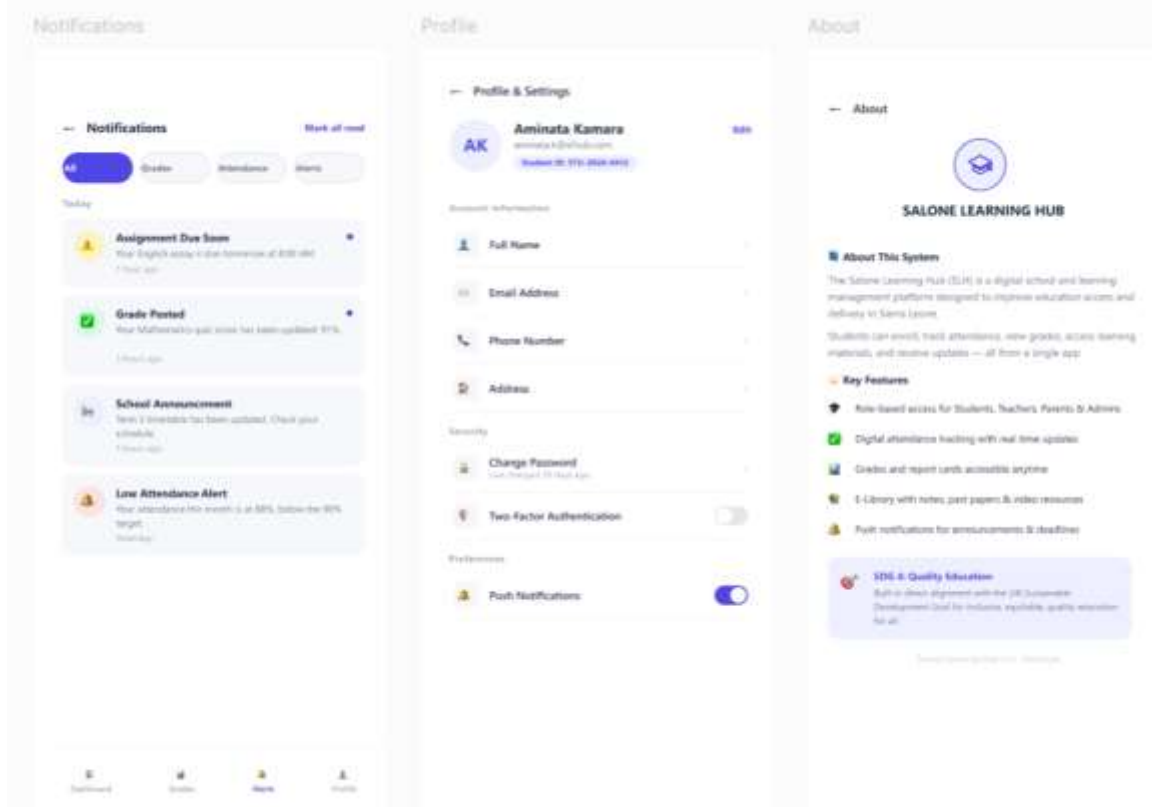
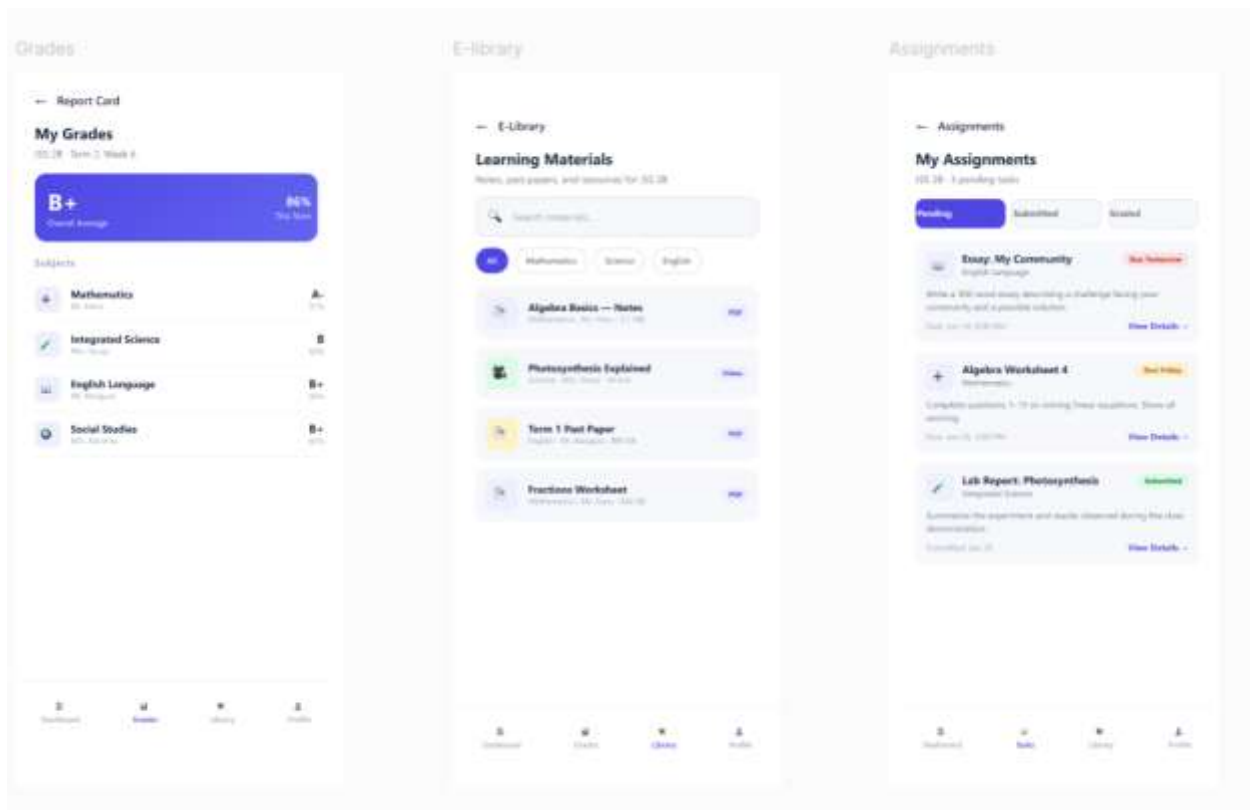
HOME ADDRESS

MEDICAL NOTES (OPTIONAL)

Upload Student's Photo

[✓ SUBMIT FORM](#)





Figma link:

<https://www.figma.com/design/5XUMmeos5JJMWDznIWH6jc/ASSIGNMENT-2?node-id=0-1&p=f&t=eCJFQQB7dniXStDA-0>

GUITHUB SCREENSHOTS

PROJECT PLAN

Below is the project plan for our "Salone Learn Hub":

1. Planning & Research
2. Assembling Requirements
3. System Architecture Design
4. Prototype Development (Figma Screens)
5. GitHub Documentation
6. Testing & Correction
7. Final Report Submission.

Below is the Activity chart for the project plan above:

CODE	Tasks	Duration [Days- Week]	Dependencies
1	Planning & Research	6 days (Week 4)	none
2	Assembling Requirements	6 days (Week 5)	1
3	System Architecture Design	12 days (Week 6&7)	1&2
4	Prototype Development (Figma Screens)	24 days (Week 7-10)	2&3
5	GitHub Documentation	18 days (Week 9-11)	3&4
6	Testing & Correction	18 days (Week 8-10)	4
7	Final Report Submission	6 days (Week 11)	3,4,5,6

Gantt Chart — Salone Learn Hub (SLH)

Below is the Gantt chart:



A Gantt chart is a visual timeline that shows when each task in a project starts, how long it runs, and how tasks overlap or follow one another.

In our chart the vertical axis lists the project tasks, while the horizontal axis represents the days/duration of the project. Each task is shown as a colored horizontal bar, the bar's position shows when it starts, and its length shows how long it runs.

The chart maps all 7 project tasks across a 72-day timeline (Weeks 4–11):

Task 1 — Planning & Research: Week 4 (Days 0–6)

Task 2 — Assembling Requirement: Week 5 (Days 6–12)

Task 3 — Architecture Design: Weeks 6&7 (Days 12–24)

Task 4 — Prototype Development: Weeks 7–10 (Days 24–48)

Task 5 — GitHub Documentation: Weeks 9–11 (Days 48–66)

Task 6 — Testing & Correction: Weeks 8–10 (Days 48–66)

Task 7 — Final Report Submission: Week 11 (Days 66–72)

Tasks 1–4 run sequentially, each starting only after the previous one ends. Tasks 5 and 6 runs in parallel, both occupying Days 48–66; meaning GitHub Documentation and Testing & Correction happen simultaneously rather than one after the other. The project then converges on Task 7, closing out at Day 72.

This timing matches the ES/EF values from the network diagram exactly, confirming that both project management tools tell the same story about the schedule.

Risk Management

Risk management involves identifying problems that could delay or disrupt a project before they happen, and having a plan ready to deal with them. Looking at our Gantt chart, the following risks were identified:

Scope Creep (Task 4 — Prototype Development, Weeks 7–10): This is the longest task in our chart, running from Day 24 to Day 48. The risk here is that the team keeps adding new features or screens beyond what was originally planned for SLH — such as extra dashboards or modules — stretching the task beyond Week 10 and pushing everything after it — GitHub Documentation, Testing, and Final Submission — off schedule. To manage this, we agreed on a fixed list of features (student, teacher, parent, and admin screens) before development began and will not add new one's mid-sprint.

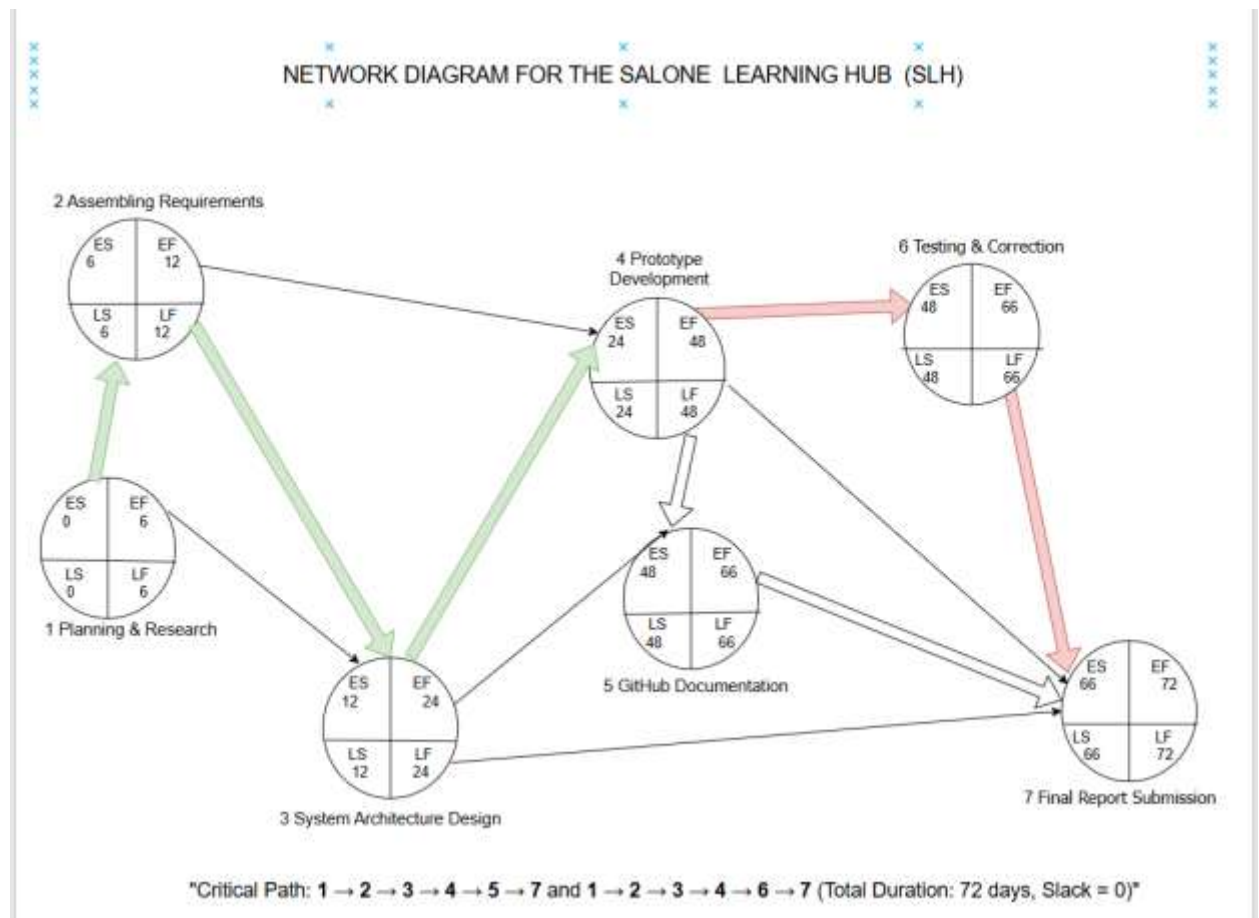
Team Member Unavailability: Since Tasks 5 and 6 (GitHub Documentation and Testing & Correction) run in parallel between Days 48–66, both require active team members working at the same time. If a member becomes unavailable during this window, both tasks are affected simultaneously. To manage this, responsibilities are shared across the group so no single task depends on one person alone.

Time Underestimation: Task 3 (Architecture Design, Weeks 6&7) and Task 2 (Assembling Requirements, Week 5) are short, back-to-back tasks. If either runs over, it delays Task 4, which is already the heaviest task. To manage this, we built no overlapping buffer between early tasks, so the team must strictly stick to each week's deadline.

Late Final Submission (Task 7 — Week 11): Task 7 sits at the very end with no buffer after Day 72. Any delay in Tasks 5 or 6 directly pushes the submission past the deadline. To manage this, we set an internal soft deadline of Day 70 for completing Tasks 5 and 6, giving a small two-day margin before the actual submission date.

Network Diagram — Salone Learn Hub (SLH)

Below is the diagram:



Our network diagram above maps out all 7 tasks of the SLH project, showing their dependencies and timing from start to finish. Each task is represented as a circle (node) divided into four quadrants displaying the ES (Early Start) and EF (Early Finish) in blue, and the LS (Late Start) and LF (Late Finish) in red/maroon. The arrows between nodes show the order in which tasks must be completed before the next one can begin.

The project kicks off with Node 1 Planning & Research (ES=0, EF=6), which feeds directly into Node 2 Assembling Requirements (ES=6, EF=12). From Node 2, the work branches one arrow goes to Node 3 System Architecture Design (ES=12, EF=24), and a shortcut arrow goes directly to Node 4, though that arrow carries slack and is not critical.

Node 3 then feeds into Node 4 Prototype Development (ES=24, EF=48), which is the project's heaviest task, running from Day 24 to Day 48.

From Node 4, the diagram splits into two parallel branches:

Node 5 — GitHub Documentation (ES=48, EF=66)

Node 6 — Testing & Correction (ES=48, EF=66)

Both run simultaneously between Days 48 and 66, before converging on Node 7 Final Report Submission (ES=66, EF=72), which closes the project at Day 72.

Since every node in our diagram has $LS = ES$ and $LF = EF$, the slack across the entire network is zero, giving us two critical paths: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 7$ and $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 6 \rightarrow 7$, both totaling 72 days. The shared trunk ($1 \rightarrow 2 \rightarrow 3 \rightarrow 4$) is highlighted in blue, the GitHub Documentation branch in white/bold, and the Testing & Correction branch in gold/yellow — making both critical paths clearly visible. The non-critical arrows ($1 \rightarrow 3$, $2 \rightarrow 4$, and $3 \rightarrow 7$) are left as plain thin lines since they carry slack of 6, 12, and 24 days respectively.

License, Design Quality & SDG Relevance

Software License

My group members and I have chosen to release the Salone Learn Hub (SLH) under the MIT License. We selected this license because it is simple, open, and perfectly suited for a Digital Public Good.

It allows schools, ministries of education, NGOs, and developers to freely use, modify, and distribute our system without legal or financial restrictions, as long as our original copyright notice is kept. Unlike the GNU GPL, the MIT License places no conditions on how others adapt or integrate our system, which means SLH can be adopted as widely as possible, maximizing its real-world impact on education delivery in Sierra Leone and beyond. Our license file is included in our GitHub repository as LICENSE.txt.

SDG Relevance — SDG 4: QUALITY EDUCATION

My group members and I designed the SLH in direct response to the educational challenges facing Sierra Leone, making it a natural fit for SDG 4: Quality Education. Schools in our country struggle daily with overcrowded classrooms, poor tracking of student attendance and performance, limited access to learning materials, and unequal access to education between urban and rural communities, all of which contribute to high dropout rates and widening learning gaps. Our system addresses these problems head-on. The digital enrolment and attendance features ensure students are properly tracked and supported, reducing dropout risk. Centralized learning materials and e-library access reduce inequality between well-resourced and underserved schools. Our digital grading and reporting system replace unreliable paper records, improving continuity and quality of academic tracking. Features like the parent portal, class scheduling, and progress notifications make education more organized and accessible to the wider community.

Together, these features make the system a practical digital tool that directly advances SDG 4's vision of inclusive and equitable quality education for all.

CONCLUSION

In undertaking this project, our group was able to apply the core principles of software engineering to a real and pressing problem affecting education delivery in our country. By adopting the Agile model, we broke down a demanding project covering system architecture, UI/UX prototyping, project planning, and documentation into manageable stages that we could design, review, and refine within the timeframe given to us.

Through the Salone Learning Hub, we designed a four-layer architecture, translated into a working Figma prototype of 21 screens covering the student, teacher, parent, and administrator experience — from onboarding and registration through to dashboards, attendance, grading, the e-library, and account settings. Our project plan, represented through a Gantt chart and network diagram, allowed us to map task dependencies and identify the critical path, while our choice of the MIT License ensures the system can be freely adopted by schools, ministries of education, and NGOs seeking to improve education access.

While SLH remains a prototype at this stage, it demonstrates a practical and scalable approach to reducing overcrowding's impact on learning, improving academic record-keeping, and making education more organized and accessible — directly supporting SDG 4's vision of inclusive and equitable quality education for all. This project has given our group valuable hands-on experience in designing, planning, and documenting a complete software system from the ground up.

REFERENCES

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R.C., Mellor, S., Schwaber, K., Sutherland, J. and Thomas, D. (2001) *Manifesto for Agile Software Development*. Snowbird, Utah: Agile Alliance.

Sommerville, I. (2016) *Software Engineering*. 10th edn. Harlow: Pearson Education.

United Nations (2015) *Transforming Our World: The 2030 Agenda for Sustainable Development*. New York: United Nations Department of Public Information.

Digital Public Goods Alliance (2022) *Digital Public Goods Standard, Version 1.1*. Oslo: Digital Public Goods Alliance Secretariat.

Pressman, R.S. and Maxim, B.R. (2020) *Software Engineering: A Practitioner's Approach*. 9th edn. New York: McGraw-Hill Education.

Nielsen, J. and Budiu, R. (2013) *Mobile Usability*. Berkeley: New Riders Publishing.